

Progressive Retrieval Attention: A Position Paper on Demand-Driven Context Expansion

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

September 12, 2026

Abstract

Large language models commonly expose non-parametric information as a flat prompt even when only a small, changing subset is useful. We argue for an explicit memory interface that separates five operations often collapsed under “retrieval”: naming a source, encoding contextual state, selecting useful identities, admitting a bounded working set, and consuming that state in attention. Progressive Retrieval Attention (PRA) is one design that realizes this separation with URI-like handles, compact routing representations, and selected layer-native K/V. Its central proposition is methodological: addressable external memory should be evaluated through distinct retrieval, contextual-state, active-attention, and task-utility contracts. Controlled companion experiments show that native-state transport can remain accurate under a bounded payload, but also expose independent failures from fragmentation, routing, position handling, and memory consumption. These observations do not establish production serving economics, general multi-hop retrieval, or superiority to optimized long context, retrieval, and prefix caching. The proposal is falsifiable by matched-budget task quality, active and resident bytes, transfer and latency, cache reuse, and content-causal counterfactuals.

1 Introduction

The transformer attention mechanism made it possible for models to condition every token on every earlier token in a sequence [33]. The same mechanism also created a central systems problem: when a model needs access to many documents, code files, logs, prior decisions, or external knowledge fragments, the simplest interface is to concatenate more text into a longer sequence. Modern long-context systems have stretched this interface dramatically, but a larger window does not by itself solve the problem of *selective use*. If only a small fraction of the context is relevant at any generation step, eager inclusion wastes computation and forces the model to perform implicit search inside an expensive dense attention substrate.

PRA proposes a different working-set rule: **recent context remains directly visible at high resolution; older or external context is retained as addressable memory and disclosed in native-KV detail only when routing predicts it is useful.** Here, *URI-addressed memory* is external source state identified by a stable URI-like reference, summarized by compact routing state, and selectively materialized as layer-native K/V. Explicit handles provide external addresses; displaced prompt history and streaming rollover are implicit addressable-memory forms that use the same machinery.

Retrieval-augmented generation (RAG) addresses part of this issue by selecting external passages before generation [20, 14, 16]. However, standard RAG often makes a single retrieval decision at the boundary of the model call. The retrieved passages are still inserted as flat prompt text,

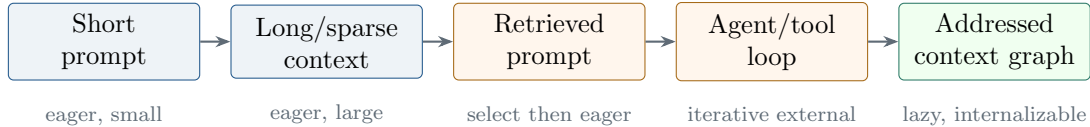


Figure 1: Context interfaces differ in when and where information becomes local to the model. PRA is a proposed design point, not a claim that the other interfaces should disappear.

and the model must consume them whether or not each passage becomes useful later. Agentic systems take the opposite approach: they inspect tool descriptions, issue searches, open files, follow references, and progressively disclose detail only when needed [38, 30]. This behavior is effective but expensive because the loop is mediated by external actions, repeated model invocations, and hand-designed orchestration.

This paper proposes Progressive Retrieval Attention as one way to internalize part of that progressive-disclosure pattern. Consider a model diagnosing a failing authentication test in a large repository. Its local prompt can advertise compact handles such as `<REF_1>`, whose local reference table maps to `repo://app/auth#module`. The runtime resolves that object, encodes it in bounded contiguous blocks, slices the resulting state into routing chunks, and exposes detailed layer-specific memory only for selected chunks. The current SDK default is one-shot; bounded memory-to-memory discovery has been studied separately, while online recursive decode-resume behavior remains a research target rather than an implied property of one forward pass.

The central question is therefore: *can a model obtain better quality per unit of active context, latency, and memory by treating external information as a progressively materialized context graph rather than an eagerly flattened token sequence?* The paper develops the vocabulary needed to answer that question, separates architectural, runtime, optimization, and evaluation claims that are often collapsed under “retrieval,” and states experiments that could reject the proposal.

1.1 Scope and Claim Types

The paper makes an architectural proposal, reports scoped implementation facts, and advances testable hypotheses. The proposal is the separation among naming, encoding, routing, admission, and native attention. Existing prototypes establish bounded source encoding, finer routing chunks, exact packed routing, row-isolated caches, one global materialization budget, and selected pretrained-family integrations. These are controlled mechanism results, not evidence of production throughput, unrestricted benchmark accuracy, recursive retrieval benefit, or serving economics. This distinction is maintained so that a working mechanism is not mistaken for a general advantage.

Figure 1 locates the proposal among neighboring interfaces. The horizontal arrangement expresses increasing separation between the local token stream and the available information; it is not a historical sequence and does not imply that PRA should replace the earlier interfaces.

The rest of the article defines the interface, its contracts, its closest alternatives, the evidence presently supporting it, and the experiments that could reject it. Detailed project chronology and publication numbering are kept outside the main argument.

2 Motivation

2.1 The Cost of Flat Context

In a decoder-only transformer with dense self-attention, the attention matrix for a sequence of length T contains $O(T^2)$ pairwise interactions per layer. Efficient kernels and hardware-aware implementations reduce constants, and sparse-attention variants reduce asymptotic or effective cost [2, 39, 18, 29]. Yet a conceptual mismatch remains: many tasks contain large amounts of potentially relevant context but only a small amount of immediately relevant evidence.

Consider a coding task. A repository may contain thousands of files. A developer rarely reads every file before making a change; they inspect a project map, open candidate modules, follow definitions, and read details near the call sites that matter. The useful information is embedded in a hierarchy: repository, package, file, class, method, block. The flat prompt interface collapses that hierarchy into a token sequence. Once collapsed, the model must recover structure through positional and lexical cues that were never designed as stable memory addresses.

The same pattern appears in scientific QA, technical support, legal analysis, and book-length reasoning. The system often knows that a reference exists before it knows whether the reference must be expanded. A long context window encourages eager expansion because the only available memory interface is token inclusion.

2.2 The Shallowness of One-Shot Retrieval

RAG retrieves documents or passages and places them in the model context [20]. This often improves factuality and makes model behavior auditable because generated answers can be traced to retrieved evidence. The limitation is that retrieval is usually performed at one granularity and one time. A retriever may return a passage that is too broad, too narrow, or missing the second-hop information required by the question. The generator then has limited ability to request a more specific subdocument without leaving the forward pass and entering an external loop.

Hierarchical retrieval partially addresses this issue by retrieving coarse summaries before detailed chunks, but the retrieved evidence is still usually materialized as prompt text. The generator receives the results of a pipeline rather than participating in a continuous memory-selection process. PRA asks whether part of this selection can be made differentiable, layer-local, or runtime-local.

2.3 The Expense of Agentic Search

Agentic systems use tools to search, read, execute, and revise. Their strength is not simply tool use; it is progressive disclosure. A capable agent first consumes compact descriptions and only loads detailed instructions, files, or logs when the task demands them. This style is naturally suited to large workspaces and multi-document tasks. Its weakness is cost and latency. Each search or file read may require a model turn, an external call, and a new prompt construction step.

The motivating question for PRA is therefore not whether agents should disappear. Rather, the question is whether the model/runtime boundary can absorb a lower-level version of progressive disclosure: reference-aware memory expansion during inference, with external agents reserved for high-level planning, verification, and action.

3 From Flat Context to Context Graphs

The repository example makes the required abstraction concrete. Before the model can route to the authentication implementation, the system needs a stable name for the object, an explicit

relation to any child objects, and a representation compatible with the layer that will consume it. This section introduces those three pieces in that order. A useful way to keep them straight is to distinguish three levels of detail. The handle is an address, the routing gist is a small representation used to decide whether a chunk looks useful, and the token-level K/V is the evidence the decoder can actually read. Moving between these levels is the “progressive” part of PRA.

3.1 Reference Handles

A *reference handle* is a lightweight token or latent marker that names an external memory object without expanding it immediately. It plays the role of a typed pointer in the prompt: the pointer says where the object can be found, but it is not a summary of the object and does not consume the object’s full token budget. In the simplest textual form:

`<REF_1> → repo://app/auth#module`

The handle is paired with a local reference table containing at least an identifier, token, and URI, with optional summary and metadata. The local table supplies the binding: `<REF_1>` has meaning only in the example or object that advertises it. Child edges exist only through this explicit table; URI-prefix similarity is not sufficient to create graph structure. The prompt can therefore advertise a possible source without pretending that the source has already been read.

3.2 URI-Addressed Memory

PRA assumes that external memory can be addressed by URIs. The URI is the stable identity of source state, not its routing gist or transported K/V. The scheme may denote local memory, files, repository objects, search results, database rows, or runtime objects:

`mem://manual#installation.cuda.vllm`

The anchor fragment after `#` is not merely a byte offset. It can encode semantic hierarchy: document, section, subsection, paragraph, function, class, or trace event. URI addressing gives the runtime a stable object to cache and gives the model a compact symbol for a potentially large subtree of context.

3.3 Prompt History as an Implicit Reference

Explicit handles are not the only way an object can enter the context graph. Once a prompt exceeds the direct-token budget D , its older prefix can be reclassified as a request-local object while the most recent suffix remains in causal self-attention. For token sequence $x_{1:N}$ with $N > D$, the implemented split is exact:

$$z_{\text{head}} = x_{1:N-D}, \quad x_{\text{direct}} = x_{N-D+1:N}. \quad (1)$$

The head is registered under `pra://implicit/prompt/head` (shown in diagnostics as `#_head`), encoded once as causal history, sliced into independently routable native-K/V chunks, and routed through the ordinary PRA memory path. No visible marker is inserted into the suffix, and no decode–re-encode round trip changes the token identities. The reserved URI is scoped to one batch row and one request; a new request replaces it rather than inheriting stale prompt history.

This mechanism turns chronological displacement into the same named-object interface used for external documents. It is not equivalent to keeping the entire prefix in dense self-attention:

the head becomes non-causal memory after it has been observed, and only selected chunks are transported. The implementation also retains `truncate` and `error` overflow policies. During generation, expired routing-sized prefixes of the direct suffix are appended to the same implicit object and re-encoded through the bounded source path. This correctness-first rollover rebuilds the head cache; incremental append remains future systems work.

3.4 Logical Context Under a Native Bound

PRA separates the size of its logical address space from the maximum context processed by the underlying model in any one operation:

$$L_{\text{logical}} \gg L_{\text{model,max}} \geq L_{\text{encode}} > L_{\text{route}}. \quad (2)$$

Long sources are encoded in bounded, optionally overlapping blocks. Context-only overlap is discarded before each block’s native K/V is sliced into smaller routing chunks. After routing, all candidate sources share one score-ordered whole-chunk admission budget:

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{model,max}}. \quad (3)$$

This does not make the base model natively support arbitrary context. It exposes a larger logical context while keeping each encoding and attention operation within a configured native limit.

Logical context may exceed native model context, but no underlying model operation may exceed the configured native limit. This is a runtime contract, not a claim that the base model’s positional or attention machinery has been enlarged.

3.5 Recursive Anchors

This subsection describes the next-stage architecture, not the current one-shot algorithm. Many references are best exposed recursively. A root object may advertise child anchors, and those child objects may advertise more specific anchors:

```
mem://manual#root → mem://manual#installation → mem://manual#installation.cuda
```

Recursive anchors would turn context into a traversed graph rather than a bag of chunks. Expansion must be child-first and bounded by shared depth, child, reference, token, latency, and byte budgets. Build states must prevent partially encoded cyclic entries from becoming searchable, while content/model/tokenizer/routing fingerprints protect cache identity. The current runtime can resolve bounded child tables during cache construction, but it does not yet use generated evidence to perform online decode-resume expansion. Paper 2.5 evaluates bounded associative discovery over a frozen memory graph, but it does not turn recursive page faults into a production decoding policy.

3.6 Layer-Specific Memory Cache

If an external fragment is encoded into memory for attention, the representation should respect the transformer’s layer structure. A key/value pair computed for one layer is not generally interchangeable with another. In practical terms, PRA does not encode a chunk once and hand the same vector to every layer. It runs the chunk through the same model stack used by the prompt and captures the representation appropriate to each configured depth. Four sets must be named

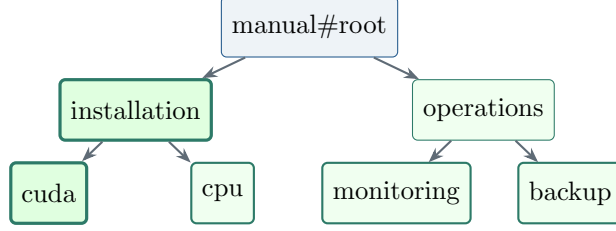


Figure 2: Future recursive anchor expansion. A coarse handle exposes children; an iterative policy would materialize the highlighted path to greater depth. This is not the current one-shot algorithm.

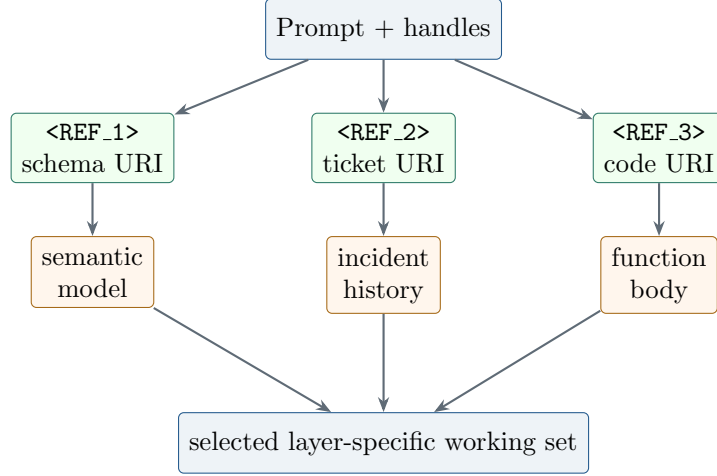


Figure 3: A context graph separates compact handles, addressable objects, and the active layer-specific working set.

separately: address layers retain compact states, detail layers retain full source K/V, routing layers score memory, and consumer layers admit selected K/V to attention. PRA therefore uses a role-aware layer-specific memory cache:

$$\text{cache}[u, c, \ell] = (K_{u,c,\ell}, V_{u,c,\ell}, G_{u,c,\ell}^K, G_{u,c,\ell}^V), \quad (4)$$

where u is a URI, c a deterministic chunk, ℓ a transformer layer, (K, V) is present only for $\ell \in L_{\text{detail}}$, and $G^K, G^V \in \mathbb{R}^{G_c \times d}$ are paired contextual address gists for $\ell \in L_{\text{addr}}$. Mean, last-token, reference-end, and GRU modes produce $G_c = 1$; k-means, prototype, self-organizing-map, and hybrid modes may retain several local regions. A gist is an index entry, not a compressed replacement for the chunk: it is cheap to compare with a query, while the full K/V preserves token-level detail after selection. This distinction matters because lower layers may encode lexical and syntactic information while higher layers may encode task-specific abstractions. Raw token embeddings or document summaries are not substitutes for layer-specific routing and attention memory.

4 Progressive Retrieval Attention

4.1 Conceptual Computation

Return to the authentication example. Local self-attention relates the test tokens and the code already present in the prompt. The memory branch answers a different question: given the current diagnosis, which advertised module chunks should become additional evidence at this layer? The computation below separates the cheap ranking operation from the more expensive attention over detailed evidence. Operationally there are two phases. A preparation phase resolves each advertised URI, partitions its content, and builds layer-specific chunk memories. A use phase lets the live prompt query the small chunk gists, materializes only the selected K/V, and fuses the resulting memory read with ordinary causal self-attention.

Let $X \in \mathbb{R}^{T \times d}$ contain the d -dimensional hidden states for T local tokens at layer ℓ . Standard causal self-attention first projects these states into queries Q , keys K , and values V . Each head has width d_h , and M masks future positions:

$$\text{SelfAttn}(X) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_h}}\right)V, \quad (5)$$

The output mixes values from preceding local positions according to query–key compatibility. A memory-consuming layer preserves that path and adds memory selected from a cache. For routing, it uses the projected query at the most recent valid position, merges its heads into $q_{t,\ell}$, and compares it with every projected-key gist $g_{u,c,\ell,j}^K \in G_{u,c,\ell}^K$ for chunk c of object u :

$$q_{t,\ell} = \text{mergeheads}(XW_Q^{(\ell)})_t, \quad (6)$$

$$a_{u,c,\ell,j} = \cos(q_{t,\ell}, g_{u,c,\ell,j}^K), \quad (7)$$

$$a_{u,c,\ell} = \text{Agg}_{j=1}^{G_c} a_{u,c,\ell,j}. \quad (8)$$

The default aggregation is maximum; mean and log-sum-exp are alternatives. URI-level gist sets can likewise select references before chunk scoring, so “reference-first” is a real two-stage route rather than a relabeling of flattened chunk scores. The aggregate score $a_{u,c,\ell}$ ranks small routing representations rather than the full chunk. Hierarchical routing aggregates chunk scores to select at most k_r references, then independently selects at most k_c chunks inside each selected reference. Content-derived gists are the default. Summaries, when available, may replace, augment, or compete with content scores under an explicit policy, but missing summaries fall back to content. Let $\mathcal{A}_{t,\ell}$ be the selected reference–chunk pairs. This routing score answers “which drawers should be opened?”; it is not an attention weight over the evidence inside a drawer. Canonical transport prepends selected historical keys and values to the local keys and values:

$$\text{PRA}_\ell(X) = W_O^{(\ell)} \text{softmax}\left(\frac{Q^{(\ell)}[K_{\mathcal{A},\ell}; K_{\text{local}}^{(\ell)}]^\top}{\sqrt{d_h}} + M_{\text{causal}}\right)[V_{\mathcal{A},\ell}; V_{\text{local}}^{(\ell)}]. \quad (9)$$

Here $K_{\mathcal{A},\ell}$ and $V_{\mathcal{A},\ell}$ concatenate full layer- ℓ token memory from selected chunks only. Memory and local positions compete in one normalization, and the layer uses its existing W_O ; there is no memory scale or new transport projection. In the running example, the gist may select the password-validation chunk, but the decoder receives that chunk’s token-level keys and values and can assign different attention weights to its individual tokens. Routing representations are therefore cheap selectors, not compressed substitutes for transported evidence.

Routing gists are selectors; selected chunks materialize full native token-level K/V. Consequently gist count, stored source K/V, transferred K/V, and active attention K/V are different accounting quantities.

The central research problem is selection: how should $\mathcal{A}_{t,\ell}$ be chosen? Possible triggers include contextual gist similarity, attention to explicit reference-token positions, uncertainty in the next-token distribution, learned gates, or runtime policies that trade accuracy against latency. Reference ranking, chunk ranking, and materialization must remain separately measurable.

4.2 PRA as Architecture, Runtime, and Research Program

The term PRA is intentionally broader than one neural module. As an *architecture*, it denotes attention layers that consume external reference memory through native K/V injection. Optional adapted modes may use cross-attention or adapters, but they are secondary variants rather than the definition of PRA. As a *runtime*, it includes the resolvers, caches, URI stores, expansion policies, and schedules that decide when and how the memory is constructed. As a *research program*, it includes the benchmarks, ablations, scaling studies, and analyses needed to test demand-driven context expansion.

These scopes support different conclusions. A standalone prototype can validate cache construction and attention mechanics without showing readiness for a production-scale pretrained model. Runtime experiments can study scheduling and K/V reuse without settling how to train the selector. Scaling studies can ask which task distributions benefit from context graphs without committing to one resolver or cache implementation. The following commitments keep those conclusions separable.

5 Conceptual Commitments

PRA is most useful as a research frame when several concerns remain separate. Collapsing them produces systems that may work but cannot be interpreted.

5.1 Naming Is Not Materialization

A reference handle states that a memory object exists and can be addressed. It does not imply that the object’s text, embeddings, or K/V tensors have already been loaded. This distinction allows a runtime to expose a large logical context graph while maintaining a small active physical context. It also supports late binding: two deployments may resolve the same logical URI through different stores, permissions, or freshness policies.

The separation creates a useful sequence of representations:

$$\text{handle} \rightarrow \text{metadata} \rightarrow \text{optional summary} \rightarrow \text{text/chunks} \rightarrow \text{layer memory}. \quad (10)$$

Each transition has a cost and can be denied, cached, audited, or deferred. Progressive disclosure is therefore not merely truncation; it is movement through explicit levels of materialization.

For a reader following one reference end to end, the sequence is concrete. The prompt first carries a symbolic address. Resolution turns that address into an authenticated, versioned object. Encoding creates small layer-specific gists plus detailed token K/V. Routing scores the gists, and only then does memory attention consume selected K/V. A system may prepare some of these representations eagerly or retrieve them from a cache, but the logical boundaries remain the same.

5.2 Selection Is Not Transport

Selecting a URI and transporting its representation into a model are different operations. Selection may use lexical retrieval, vector similarity, explicit handle positions, learned gates, uncertainty, or an oracle. Canonical transport uses native layer K/V; prompt concatenation, cross-attention, and adapters are comparison transports. An experiment that changes both at once cannot determine whether a gain comes from finding better evidence or from presenting the same evidence through a better neural interface.

PRA evaluations should first bypass selection: restoring all historical native K/V must match full self-attention when positions and masks are equivalent. Oracle sparse selection then measures the cost of omitting K/V, and only afterward should routed, shuffled, and learned selection be evaluated. Cross-attention remains a separate adapted-transport comparison.

This sequence supports a quantitative error decomposition. The gap from full SelfAttention to native all-memory measures transport, partition, and position cost; the gap from all-memory to sparse oracle measures sparsification cost; and the gap from oracle to routed memory measures selection cost. For condition c , Recovered Context Benefit is

$$\text{RCB}_c = \frac{L_{\text{tail}} - L_c}{L_{\text{tail}} - L_{\text{full}}}. \quad (11)$$

RCB asks how much of the predictive advantage of dense context is recovered over a local tail. It is undefined when dense context offers negligible advantage and is not clipped when memory is harmful or outperforms the dense baseline. The companion systems quantity is active token-K/V fraction, $(T_{\text{local}} + T_{\text{retrieved}})/(T_{\text{local}} + T_{\text{displaced}})$; routing gists are reported separately. The scaling thesis is supported only if RCB remains high while this active fraction falls, not merely when a PRA loss is lower than an unrelated baseline.

The same separation applies inside selection. Reference ranking answers which addressable objects remain candidates; chunk ranking answers which portions of those objects are materialized. Independent budgets are necessary to prevent a prolific reference from consuming every chunk slot. Reported cache size, selected gist count, and transported token count are different quantities and should never be used interchangeably.

Active K/V is not total resident K/V. Sparse attention becomes a GPU-capacity benefit only when unselected payload can remain off device, be evicted, or be shared without materialization.

5.3 Implementation Invariants

The mature implementation is easier to reproduce when its invariants are stated before its optimizations. First, when PRA is disabled or has no selected memory, the host attention path is bypassed rather than approximately reconstructed. The standalone tests require the native history replay to match full causal attention when positions and masks are equivalent; pretrained-family adapters additionally require exact PRA-off logits, hidden states, generation, and tested cache behavior. Exactness applies to the named controls, not to sparse memory as an approximation of an omitted dense context.

Second, the compact index and attention payload have different jobs. A routing gist may be a pooled hidden state, projected key, or learned semantic vector. It identifies a URI and routing chunk. Once selected, the payload is the host layer’s full-detail native K/V, including its head layout, dtype, positional treatment, and cache convention. Canonical PRA introduces no separate memory output projection or residual scale; the historical cross-attention mode does.

Third, accounting follows the lifecycle rather than one ambiguous “context fraction.” For the ordering below, all three quantities count *unique source positions at one named layer*: T_{logical} is every addressable source position, $T_{\text{requested}}$ is the union belonging to selected references or parents, and $T_{\text{materialized}}$ is the subset admitted to one active operation. Then

$$0 \leq T_{\text{materialized}} \leq T_{\text{requested}} \leq T_{\text{logical}}, \quad (12)$$

but equality is not assumed. The inequality does not apply to layer-summed token states, padded slots, physical duplicates, or bytes: each can exceed its unique-position analogue without violating the admission contract. Routing gists, resident detail K/V, transferred bytes, padded attention allocation, and cumulative attended K/V work are therefore reported separately. This prevents a logical selection rate from being presented as a physical saving.

Finally, the lifecycle remains bounded. Every source encode, direct-plus-memory attention operation, resolver expansion, transfer, and cache admission is subject to explicit token, byte, depth, identity, and isolation constraints. CPU or off-device residency reduces accelerator capacity only when selective transfer keeps unselected payload off the active device; it does not erase encoding, storage, or communication cost.

5.4 Logical Isolation Must Survive Physical Batching

Each sequence has a logical memory address space even when kernels share physical tensors. Two rows may even use the same URI string for different content; the row-local reference table decides what that name means. An efficient implementation can still run the prompt as one batch by keeping a private cache namespace for each row, routing row i only against namespace i , and masking padded memory positions. Correct batching may evaluate each row independently, pad one masked rectangle, or group rows into deterministic length buckets. These execution plans are equivalent only if empty rows receive an exact zero memory residual, outputs return to original order, and one row’s cache cannot influence another row’s logits. Padding allocation and bucket count are systems metrics, not semantic changes to routing.

5.5 Language Ability Is Not Reference Adaptation

Adding a memory path can improve a reference-formatted task while damaging ordinary language modeling. Conversely, freezing the local model can guarantee preservation while limiting adaptation. The relevant object is therefore a two-dimensional outcome: reference-task improvement and local-task retention. A useful method should report both, including the exact trainable parameter groups.

This distinction motivates staged optimization. First establish that each architecture learns ordinary language. Then initialize from the matching checkpoint and train only newly activated reference-specific modules. Finally, compare joint fine-tuning under a mixed plain/reference curriculum. Preservation obtained by freezing is a property of the optimization policy, not proof that an architecture is intrinsically immune to forgetting.

5.6 Memory Identity Requires Isolation

URI identity is part of the conditioning state. If a batched implementation combines references from unrelated examples without a corresponding mask, it changes the task and can leak evidence across samples. The standalone implementation exposed this issue directly: a batch-union cache made within-batch shuffling ineffective. Correctness required a cache per example. Efficient PRA

runtimes must preserve the same logical isolation through cache indices or attention masks even when physical storage and kernels are shared.

6 Virtual Memory for Transformers

Virtual memory changed computing by separating the address space visible to a program from the finite physical memory resident at a given moment. Pages are named independently of their current location, loaded lazily, cached while useful, and evicted under pressure. Working-set theory connects recent access behavior to resource allocation rather than pretending that every address must remain resident [8]. Foundation-model context has an analogous tension: the logical information relevant to a task can be far larger than the token and K/V memory physically active during one decoding step.

PRA proposes a *virtual context space*. URIs and anchors play the role of stable logical addresses. Resolved text or structured objects are backing-store content. Encoded layer-specific K/V entries are resident pages. Selection or gating acts as an admission policy. A missing selected representation creates a context fault that triggers resolution and encoding. The active reference cache approximates a working set, while eviction and reuse determine whether the abstraction saves end-to-end cost.

6.1 PRA as Semantic Virtual Memory

The target is not literal infinite context. It is a systems regime in which logical context is much larger than physical active context:

Logical addressable context can grow far beyond the model’s native attention window and GPU-resident K/V capacity without requiring all logical context to be simultaneously materialized in attention.

We call this *effectively unbounded logical context*, *externally backed logical context*, or *demand-loaded context*. All three phrases remain conditional on finite backing storage, an executable lookup policy, bounded native operations, and measured systems cost. They do not imply mathematically infinite memory, constant lookup time, or quality that is independent of corpus scale.

For one concurrently active request, the auditable peak-device target is a maximum over time of non-overlapping accounting categories. Every M term below is device bytes at time t :

$$\begin{aligned}
 M_{\text{GPU,peak}} \approx \max_t [& M_{\text{model}} + M_{\text{direct}}(t) + M_{\text{index}}(t) \\
 & + M_{\text{resident detail}}(t) + M_{\text{active extra}}(t) \\
 & + M_{\text{workspace}}(t) + M_{\text{transfer}}(t)].
 \end{aligned} \tag{13}$$

where M_{direct} is live prompt/history K/V, M_{index} is routing and address metadata resident on device, and $M_{\text{resident detail}}$ is persistent reusable source K/V. $M_{\text{active extra}}$ contains only attention-specific allocations not already counted as direct or resident detail—for example a gathered K/V copy or padding; a zero-copy view contributes no second copy. $M_{\text{workspace}}$ is kernel scratch, and M_{transfer} is non-overlapping staging storage present at the same instant. Allocation ownership must make these categories disjoint; one tensor cannot be charged twice merely because it is both resident and attended. These terms are peak co-resident allocations, not a sum of separate per-stage maxima or per-layer traffic; cumulative K/V copies and attention work are separate counters. The intended signature is that active-state and workspace cost depend primarily on the task working set. Index

memory, backing-store memory, and routing work may still grow with the corpus, resident detail depends on cache policy, and a poor selector may broaden the working set. Scale-independent physical memory is therefore an experimental target, not a present claim.

Later results refine the early “chunk equals page” analogy into three independently configurable granularities:

$$G_{\text{encode}}, \quad G_{\text{search}}, \quad G_{\text{materialize}}. \quad (14)$$

G_{encode} is the context used to create meaningful representations; G_{search} is the conceptual routing or native-graph node; and $G_{\text{materialize}}$ is the actual native-K/V interval loaded into active attention. Coarser encoding can contextualize finer search nodes, while physical materialization may later union or window several selected nodes. Equality is allowed when a workload does not benefit from separating them. Existing traversal experiments show why the choice matters: finer search nodes can reduce conceptual payload while also removing useful associative edges.

Sparsity also has a layer axis. PRA may be sparse over

$$\text{memory regions} \times \text{token/K/V detail} \times \text{consumer layers}. \quad (15)$$

Paper 2 establishes selective pretrained-model consumer paths. Corrected Paper-3 replay and Paper-4.5 cross-model calibration now show why neither “always all layers” nor “always the last eight” is a stable rule. Placement depends on the model, workload, physical materialization, transport, and optimization objective. All eligible consumers remain a conservative reference-correctness fallback when sparse consumption has not been validated; it is not a claim of quality optimality.

6.2 Current Evidence and Its Boundaries

Table 1 freezes the evidence used by this article as of 11 September 2026. It is a map from architectural propositions to scoped observations, not a claim that the observations are independent replications. Detailed campaign status, artifact paths, and chronology remain in the versioned companion ledger `virtual_memory_status.json` and the article’s claim ledger.

6.3 Address Spaces, Protection, and Lifetime

A useful virtual context address is more than a string. It should identify scheme, authority, object, semantic anchor, version, tenant, and access policy. Two identical texts with different provenance may not be interchangeable; one URI whose content changes over time should not silently reuse stale K/V. Cache keys may therefore require a tuple such as

$$(u, v, \ell, m, \theta, p), \quad (16)$$

where u is URI, v content version, ℓ layer, m model identity, θ relevant encoder parameters, and p policy domain. Cache lifetime can range from one sequence to a user session, repository revision, model deployment, or immutable corpus version. Longer reuse improves amortization but increases freshness and revocation risk.

Permissions must be enforced before resolution and again when resident memory is shared. A model should not infer a protected reference because another tenant populated the same physical cache. Logical address spaces and attention masks must survive batching, deduplication, paging, and distributed placement.

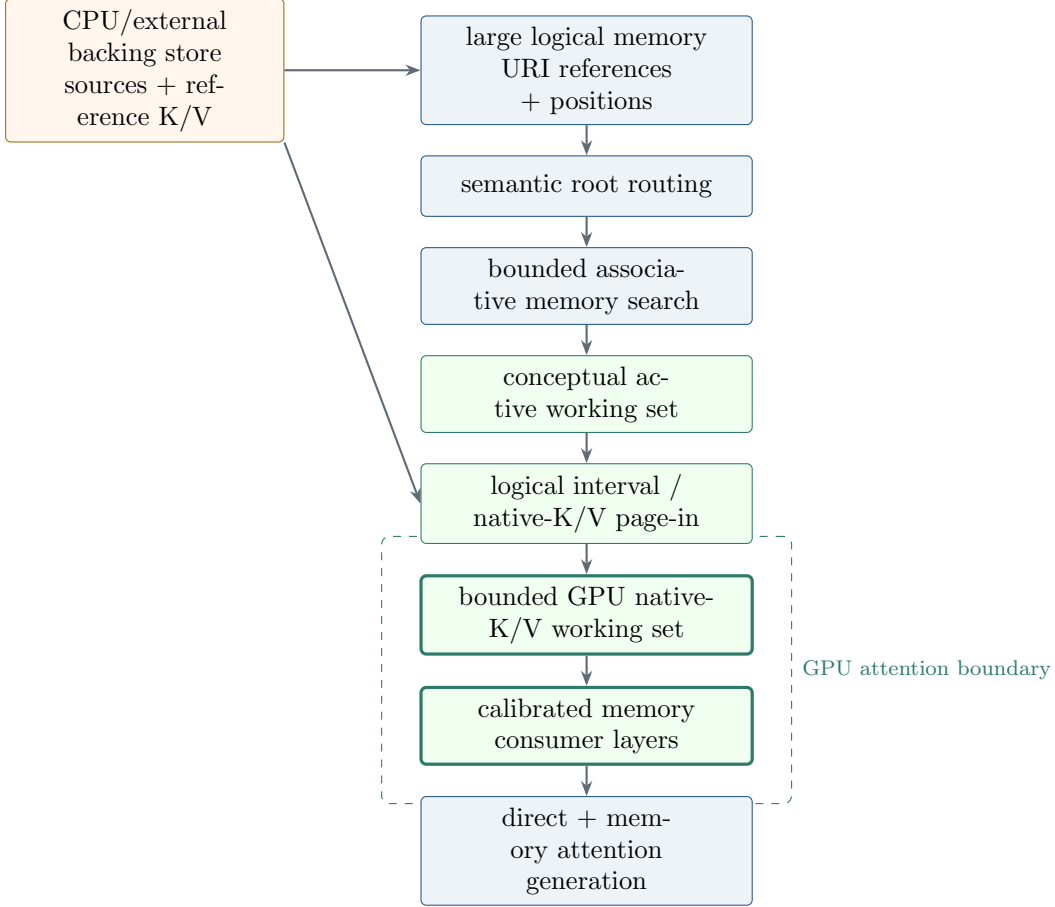


Figure 4: Semantic virtual-memory target. Logical memory and its backing store remain outside the GPU attention boundary. Root routing and associative search choose a conceptual working set; only the final native-K/V intervals and consumer layers determine active attention state.

6.4 Page Faults and Progressive Disclosure

A conventional page fault is generated by an exact address access. A model’s “context fault” is less crisp: it may be inferred from explicit handle attention, uncertainty, a learned router, or a runtime policy. False negatives omit evidence; false positives waste resolution and encoding. This turns fault prediction into a joint learning-and-systems problem. Recursive anchors further resemble multi-level page tables only loosely: following a child changes semantic granularity rather than merely translating an address.

6.5 Where the Analogy Breaks

CPU pages are intended to preserve exact program semantics regardless of placement. Neural memory is approximate, content-dependent, and transformed by model parameters. Loading a document into K/V does not guarantee that the model will use it correctly. Attention can blend untrusted entries, positional semantics may matter, and encoding may be more expensive than transport. The virtual-memory analogy is therefore valuable for identifying address, isolation, admission, working-set, and eviction problems; it is not evidence that operating-system policies can be copied unchanged.

Table 1: Evidence supporting the position, with counterexamples and unresolved tests kept equally visible. Results are companion measurements rather than original experiments of this position article.

Architectural proposition	Strongest supporting observation	Counterexample or boundary	Unresolved test
Contextual native state can be selected under a bounded payload	Encode-once slicing preserves the controlled transport benefit through the split-256 condition (255 addressable source units) while fixed- k active fraction falls to about 5%.	Annotated target coverage remains incomplete, so preserved task loss is not proof of natural semantic retrieval.	Matched long-context, selected-text, and native-K/V execution on natural tasks.
Position is part of memory identity	A controlled attention operator is invariant to common translation when relative geometry is unchanged; tested reset and distance policies show scoped, policy-dependent recovery.	Correct transport does not make semantic routing and attention geometry interchangeable.	Frozen cross-family replication under the same positional interventions.
Discovery and consumption are separate contracts	Bounded associative traversal can recover annotated descendants and exposes native Q/K topology.	Better traversal does not reliably improve generated answers; entry competition and broad activation remain harmful.	Learned entry, traversal, and consumption on an untouched task cohort.
Logical selection is not physical reuse	Selected K/V can remain off device or be attached from a resident cache without re-encoding in scoped prototypes.	Transfer, cache churn, temporary buffers, or low reuse can erase the theoretical saving.	Concurrent serving with eviction, cancellation, measured hit rates, and end-to-end cost.

Classical translation starts from an exact virtual address and follows deterministic page tables. PRA instead separates three operations. *Resolution* maps an advertised URI and version to an object and can be exact and deterministic. *Semantic selection* predicts which objects or regions are useful for the current query and may be learned or heuristic. *Speculative expansion* proposes related descendants or neighbors before their usefulness is known:

$$\begin{aligned} &\text{question} \rightarrow \text{semantic candidates} \rightarrow \text{exact URI resolution} \\ &\quad \rightarrow \text{optional expansion} \rightarrow \text{admitted native K/V.} \end{aligned} \tag{17}$$

Selection and expansion therefore have semantic failure modes with no direct analogue in ordinary virtual memory: a wrong root, distractor competition, a missing native edge, over-broad activation, or a miscalibrated controller. Resolution failure, stale versions, and authorization failure are different address/lifecycle errors. Moreover, a published cache may already contain every source K/V tensor, in which case a “miss” means admission or transfer, not cold encoding. Only an absent compatible representation triggers source encoding. Eagerly publishing all K/V and later selecting transfers is evidence about admission and reuse, not evidence that cold source encoding itself is demand driven. Associative prefetch is useful only when recall, false activation, and physical cost are measured together.

6.6 Decisive Virtual-Memory Scaling Experiment

The unresolved systems claim requires a direct scaling experiment, not extrapolation from small fixed- k curves. Construct matched logical memories of 32K, 128K, 512K, 2M, and at least 8M tokens while holding task families and evidence requirements constant. At every scale report: answer quality; root and evidence recall; active native-K/V tokens and bytes; peak HBM; CPU/reference-store memory; routing, index, and graph-search latency; interval-gather and H2D latency; TTFT; TPOT; and, when the runtime supports it, throughput and concurrency. Dense native context, sparse attention, RAG, and agentic search should receive matched task and hardware conditions.

The desired signature is stable matched-task output quality while logical memory grows from 32K to 8M+ tokens and the GPU native-K/V working set remains approximately bounded by a task-dependent budget. Search or index latency may grow, and backing-store memory should grow; the narrower hypothesis is that GPU attention working-set cost does not grow linearly with total logical context. Failure to preserve recall, quality, or bounded active K/V at scale would narrow or reject the virtual-context thesis. Until this experiment exists, PRA does not provide scale-independent physical memory, constant-cost page faults, or reliable routing at arbitrary memory size.

7 Current Architecture and Reference-Memory Lifecycle

A PRA runtime is a lifecycle rather than a single retrieval call. The current one-shot path is:

1. **Advertise:** expose handles, summaries, types, provenance, and access policy.
2. **Select references:** rank candidate handles for a sequence, position, layer, or decoding interval.
3. **Resolve:** map selected URIs to immutable or versioned content.
4. **Encode:** process source text as bounded contiguous blocks and retain host-native, layer-specific K/V; context-only overlap is not a separately addressable payload.
5. **Slice and index:** divide encoded K/V into smaller routing chunks and derive one or more compact gists without replacing the detailed payload.
6. **Route:** rank reference and chunk identities against the compact index.
7. **Admit and transfer:** choose whole chunks under one native-operation budget and move only selected K/V from CPU or other backing storage when needed.
8. **Attend:** allow authorized sequence positions and layers to consume the memory.
9. **Reuse or evict:** retain useful entries across tokens or requests, or release them under pressure.
10. **Audit:** record which version, URI, layer, and policy influenced the output.

This lifecycle suggests a cost objective. For task quality Q , prompt tokens T_p , encoded reference tokens T_r , latency L , and active cache bytes B , a system should be compared on a frontier such as

$$\max Q \quad \text{subject to} \quad c_p T_p + c_r T_r + c_l L + c_b B \leq C. \quad (18)$$

The constants depend on hardware, cache reuse, and service-level objectives. PRA is not successful merely because it shortens the visible prompt. Reference encoding that is repeated for every request may cost more than eager context. The economic case depends on selective admission, amortized reuse, or better quality under a fixed resource budget.

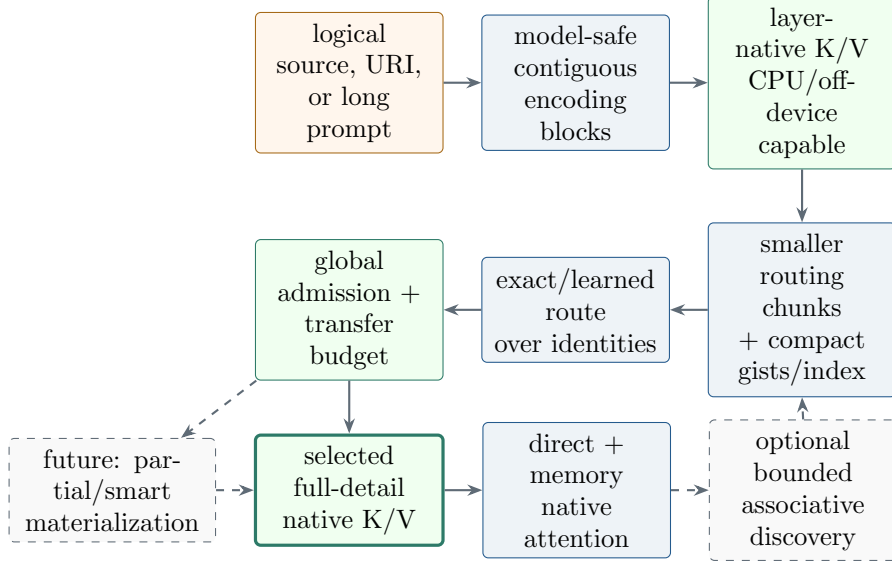


Figure 5: Canonical PRA architecture. Solid arrows are the one-shot whole-chunk SDK path. The compact index selects identities; attention consumes selected full-detail, layer-native K/V. The dashed discovery loop is an optional experimental extension, not the one-shot default; partial materialization remains a separate design choice.

8 Layer Roles and Calibrated Profiles

8.1 Four roles, not one “PRA layer” set

Depth participates in four different operations:

$$L_{\text{addr}} : \text{retain compact states that can address memory,} \quad (19)$$

$$L_{\text{detail}} : \text{retain full source-native K/V for possible page-in,} \quad (20)$$

$$L_{\text{route}} : \text{execute reference or chunk selection,} \quad (21)$$

$$L_{\text{consume}} : \text{join selected memory K/V to direct attention.} \quad (22)$$

These sets may overlap but are not synonyms. An address-only layer can retain a cheap gist without retaining token-level detail. A route-once layer can choose identities later consumed by several layers. A detail layer can remain dormant until another layer’s routing decision materializes its K/V. A consumer requires corresponding detail K/V, so $L_{\text{consume}} \subseteq L_{\text{detail}}$ in the current runtime. This vocabulary replaces the ambiguous phrase “PRA layers” throughout the series.

The stable implementation invariant is

$$\text{correct positional transport} + \text{coherent materialization} + \text{sufficient consumer depth.} \quad (23)$$

None of the three terms repairs another. A strong layer profile cannot rescue K/V replayed in an incompatible RoPE frame. Exact evidence cannot rescue a consumer topology that amplifies interference. Conversely, all-layer consumption can be transport-correct while lowering quality.

8.2 Calibration without test-set selection

Layer profiles are policies, not architectural constants. A candidate profile must be selected on a validation cohort with the model, tokenizer, workload, materialization policy, and cost definition

frozen; the final quality and cost estimate must then use untouched test identities. Reference-correctness activates every eligible consumer as a conservative fallback when no sparse profile is qualified. Quality-max, balanced, and economy profiles denote different predeclared objectives on the validation frontier, never optima selected from the reported test set. Every registry entry must record provenance, uncertainty over independent identities, and a safe override.

9 Relationship to Existing Work

PRA draws mechanisms from several mature lines of work. The comparison is organized by *which resource changes*: prompt text, token connectivity, positional range, retained K/V, persistent state, retrieved neural state, or the external action boundary. This framing narrows the novelty claim to an access contract that combines stable names, a compact semantic index, selective native-K/V materialization, and a bounded lifecycle. None of those ingredients is claimed as individually new.

9.1 Retrieved Text and Latent Retrieval

RAG, REALM, and dense passage retrieval select external text, usually before generation, and let the generator encode the selected passages in its current prompt [20, 14, 16]. Their changed resource is the input text. This preserves a simple model interface and lets retrieval scale independently, but selected text still incurs prompt encoding and dense participation once inserted. PRA can use the same retriever to choose candidate URIs, then route smaller chunks and consume cached layer-native state rather than append every selected passage.

Latent retrieval moves selection closer to model representations. The k NN-LM interpolates the model distribution with neighbors from a hidden-state datastore; Memorizing Transformers retrieve prior token memories; and RETRO retrieves text chunks for learned generator-side conditioning [17, 34, 3]. These methods and PRA all expose non-parametric information during inference. Their memory values and fusion rules differ: PRA retains URI identity and provenance, uses compact vectors only for search, and presents selected full-detail native K/V to ordinary attention. It does not interpolate a neighbor token distribution or require RETRO’s trained cross-attention architecture.

9.2 Sparse, Local, and Block Attention

Longformer, BigBird, Reformer, and Routing Transformer change token-to-token connectivity inside an admitted sequence through local windows, global tokens, hashing, or content routing [2, 39, 18, 29]. These methods answer how a long sequence should interact after it is present. PRA first asks which external state should enter the active operation. The approaches are complementary: a sparse or block kernel could execute direct-plus-materialized K/V after PRA admission.

Landmark Attention similarly gives a long sequence coarse random-access landmarks [26]. A PRA gist has a related indexing role, but it belongs to a separate addressable cache and is not the evidence consumed by attention. The selected routing chunk materializes its detailed K/V payload. This routing–payload distinction is the central difference.

9.3 Long-Context Position and Sequence Range

Context-extension methods change the length over which one model invocation can preserve useful token interaction. Transformer-XL adds segment recurrence and relative positions; Position Interpolation and LongRoPE extend the operating range of positional geometry [6, 4, 9]. Their strength

is continuous access to a large admitted sequence. PRA does not enlarge the host model’s native limit. It constructs logical scope from model-safe encoding blocks and admits a bounded subset to each operation. A longer native window can therefore improve PRA’s encoding and direct context while leaving the logical-versus-active separation useful.

9.4 KV Retention, Compression, Eviction, and Quantization

H₂O and SnapKV evict or retain decoding-cache tokens, ClusterKV merges cache state in a semantic space, and KIVI reduces bytes through asymmetric quantization [40, 21, 23, 24]. Their changed resource is an already-created KV cache. PRA instead chooses provenance-bearing reference and chunk identities before their payload enters active attention. These techniques are compatible: eviction, merging, quantization, or future token-level materialization can operate inside PRA’s selected working set. Their errors must then be measured separately from routing errors.

9.5 Recurrent, Compressive, and Persistent State

Compressive Transformers, Infini-attention, TransformerFAM, Titans, and selective state-space models carry forward compressed, recurrent, feedback, learned, or fixed-width state [28, 27, 15, 1, 13]. StreamingLLM preserves a chronological attention working set through sinks [35]. These methods change the state propagated through time and can be more economical than retaining addressable token detail. PRA preserves separately named source objects and can revisit selected full-detail K/V out of chronological order. Its implicit prompt head is the closest overlap, but beyond the native horizon its bounded-block encoding is an explicit approximation rather than a sufficient recurrent state.

9.6 Prompt and Context Caching

PagedAttention manages serving-time KV allocation, while Prompt Cache reuses schema-named prompt modules and CacheBlend selectively recomputes cached chunks to restore missing cross-chunk context [19, 10, 37]. RetrievalAttention instead retrieves a sparse subset of the long-context KV cache through an attention-aware vector index [22]; it is the closest direct comparison to PRA’s selected native state. Its optimized serving path targets long-context attention, whereas PRA additionally requires persistent object identity, explicit admission/materialization contracts, and optional hierarchical expansion; the present PRA prototypes do not match its production-scale runtime evidence. These systems change where prompt state resides and how much prefill is repeated. PRA shares their concern with positional validity, reuse, transfer, and backing storage, but adds semantic routing over addressable objects and a hard active-K/V admission budget. Conversely, PRA does not yet provide their serving kernels or throughput evidence; an optimized runtime should build on these systems rather than treat CPU residency alone as a speed result.

9.7 Memory-Augmented Networks

Neural Turing Machines and end-to-end Memory Networks couple a controller to differentiable external memory and demonstrate content-addressed reads, writes, and multiple hops [12, 31]. Hierarchical Attention Networks and adaptive computation likewise motivate multi-level representation and variable computation [36, 7, 11]. PRA inherits the general principle that access can be learned and iterative. It differs in preserving host-transformer-native K/V, stable URI identity, cache provenance, and an exact disabled path. Paper 2.5 tests parameter-free bounded associative

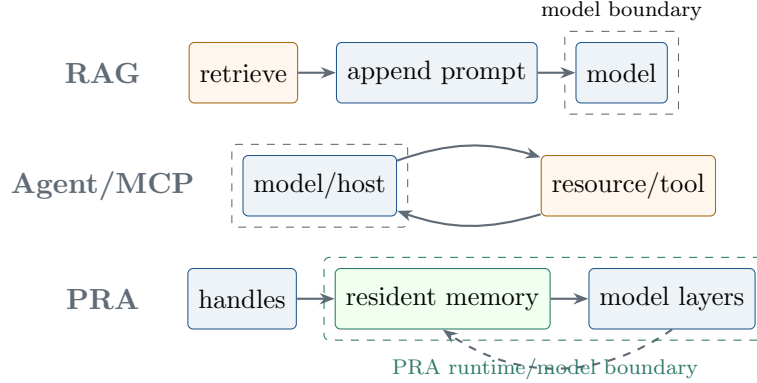


Figure 6: RAG selects before a model call, agents and MCP iterate across an application boundary, and PRA proposes addressable memory consumption within a runtime/model boundary.

discovery, but it is not equivalent to the trained multi-hop controllers in those systems and does not establish reliable online fault handling.

9.8 Agentic and Protocol-Level Access

ReAct and Toolformer coordinate reasoning with external actions [38, 30]. They change the action boundary and can search, verify, or modify the world across model calls. PRA targets a narrower read path for stable or versioned resources. It does not replace explicit tools, side effects, planning, or verification.

9.9 MCP and IDE Assistants

The Model Context Protocol (MCP) defines a client–server interface through which AI applications discover resources, prompts, and tools [25]. IDE assistants similarly expose repository search, symbol navigation, diagnostics, terminals, and version-control state. These systems make external capability discoverable, but the host application still decides how retrieved resource content enters a model context. PRA is complementary: MCP can supply the address space and resolver boundary, while PRA can define how selected resources become internal attention memory.

This combination suggests a layered interface. MCP or an IDE host handles authentication, capability discovery, transport, and side effects. A PRA runtime handles content identity, versioned encoding, cache admission, and model-local use. High-level tool calls should remain explicit and auditable; repeated read-only access to stable resources is the more plausible target for latent memory.

10 Embodied, Situated, and Referential Memory

Embodied and enactive approaches argue that cognition is shaped by recurrent interaction among brain, body, and environment rather than by isolated symbol manipulation alone [32]. The extended-mind thesis further asks when reliably available external artifacts participate in a cognitive process rather than merely serving as passive background [5]. PRA does not imply that a transformer is embodied or that neural attention reproduces human memory. These traditions instead offer a useful design question: what changes when external structure is not compressed entirely into internal parameters, but remains available through stable, action-conditioned references?

Table 2: Conceptual foundation-model memory stack. Entries are architectural roles, not claims about current product implementations.

Layer	Typical content	Lifetime	Access mode
Parametric memory	linguistic and learned regularities	model version	dense forward pass
Working memory	current goals, dialogue, active evidence	sequence/session	local attention
Episodic memory	prior interactions and outcomes	user/project history	event retrieval
Semantic memory	documents, schemas, concepts, policies	corpus version	addressed retrieval
Procedural memory	tools, workflows, APIs, permissions	deployment	explicit action
World-state memory	environment, simulations, live systems	dynamic	observation/model update

Human activity routinely relies on referential scaffolding. A person remembers that evidence is in a notebook, a clause is in a contract section, a procedure is in a manual, or an explanation belongs to a colleague. The remembered address, cue, and access routine can be more useful than a lossy internal copy. Retrieval is often situated: the object sought depends on the current task, physical or social environment, and sequence of prior actions. Hierarchical cues support progressive refinement from place or document to section and detail.

PRA translates only the computational aspects of this observation. Reference handles are cues; URIs stabilize identity; resolvers implement access routines; routing gists and optional summaries support selection; and layer-specific caches provide a temporary active representation. Recursive anchors make access path-dependent. The analogy predicts that a useful model memory system should preserve provenance and affordances, not only vector similarity. It also predicts failure when the environment is unreliable, addresses become stale, or access policy is ignored.

This perspective broadens evaluation beyond factual recall. A referential memory architecture should be tested on navigation, coordination, revision, and action: whether it can locate the right object, notice when it changed, follow a dependency, respect a permission, and update a plan after new evidence. Those are situated-memory behaviors, not merely next-token improvements.

11 A Future Foundation-Model Memory Stack

Foundation models are likely to require a hierarchy rather than one universal memory mechanism. Table 2 sketches distinct roles. The boundaries are conceptual and may be implemented by multiple coupled systems.

PRA is best viewed as an addressing and materialization fabric across this stack. It does not replace weights, local context, databases, tool protocols, or world models. It could allow compact working memory to refer to episodic events, semantic objects, and procedural capabilities; runtime policy would decide which representations can cross into model-local memory. Different classes require different consistency. Immutable literature can use content-addressed caches, while live operational state may require short leases and mandatory revalidation.

A complete stack also needs a control plane. It must assign identity, authenticate principals,

describe provenance, negotiate capabilities, track versions, account for cost, and revoke access. The neural data plane then consumes authorized representations. MCP provides one possible application-level discovery and transport interface [25]; PRA addresses the separate question of selective internal materialization.

12 Enterprise and Professional Applications

Enterprise information is unusually compatible with context graphs because it already has identifiers, ownership, revisions, links, and permissions. The opportunity is not to pour every enterprise object into a prompt, but to expose a governed address space whose active working set follows the task.

12.1 Software Engineering

A software task can begin from repository, branch, commit, package, symbol, test, issue, and build-log handles. GitHub objects and IDE symbol graphs provide stable navigation edges; PRA could keep interfaces and summaries active while materializing implementation bodies or historical diffs only when needed. Required controls include commit-aware cache keys, generated-file labeling, repository permission isolation, and tests that detect stale symbol memory. SWE-bench-style tasks and repository navigation benchmarks can measure whether progressive access reduces loaded tokens without hiding required dependencies.

12.2 Analytics, BI, and SQL

Data work spans warehouses, catalogs, SQL schemas, semantic layers, dashboards, lineage graphs, and business glossaries. A Snowflake table, BI metric, or transformation can be a versioned reference with ownership and freshness metadata. The model may first inspect a semantic metric definition, then expand the underlying model, columns, joins, and query history. Evaluation must penalize stale schemas, unauthorized columns, invalid SQL, and semantically wrong but executable queries. Live execution should remain an explicit tool action, while read-only schema memory is a candidate for caching.

12.3 Organizational Knowledge and Operations

Confluence pages, Jira issues, runbooks, incidents, and service catalogs form a cross-linked operational memory. Handles can preserve the distinction between approved policy, draft discussion, resolved incident, and current ticket state. Freshness and authority are central: an old runbook should not silently outrank a current service notice. Useful tasks include incident diagnosis, change-impact analysis, status reporting, and tracing a decision from ticket to documentation and code.

12.4 Scientific Literature

Scientific assistants need identity at article, version, section, figure, dataset, and claim level. Recursive anchors can move from abstract to method, result, appendix, and cited source. Provenance traces should distinguish quoted evidence from model inference. Benchmarks should test multi-paper synthesis, contradictory findings, retractions, and whether the model can expose the exact evidence path rather than only produce plausible prose.

12.5 Legal and Medical Assistance

Legal and medical domains make the risks clearest. Jurisdiction, effective date, patient identity, consent, and source authority are part of meaning, not optional metadata. PRA could help preserve these constraints through versioned, permissioned references and auditable access paths. It must not turn latent memory into an unreviewable decision channel. High-stakes outputs require authoritative-source validation, explicit uncertainty, human review, and strict separation between informational retrieval and actions affecting rights, treatment, or records. Evaluation should emphasize provenance completeness, stale-evidence detection, privacy leakage, and calibrated abstention rather than fluency.

13 Illustrative Design Space

Table 3 gives a conceptual comparison. The entries are hypotheses and design claims, not measured results.

Table 3: Conceptual comparison of context-access strategies. This table is illustrative and does not report experimental measurements.

Strategy	Context interface	Strength	Failure mode
Flat long context	Eager token sequence	Simple and general	High cost when relevance is sparse
One-shot RAG	Retrieved prompt passages	Good factual grounding	Retrieval granularity fixed before generation
Agentic search	External tool loop	Flexible progressive disclosure	High latency and orchestration cost
PRA	URI-addressed memory cache	Demand-driven latent expansion	Requires learned/runtime selection policy

14 Evaluation Doctrine

PRA requires counterfactual evaluation because a single task score cannot reveal how memory was used. At minimum, a promising checkpoint should be evaluated with valid references, a disabled reference path, shuffled references, irrelevant references, empty references, oracle references, and all evidence visible in local context. Targets and prompt structure must remain fixed across these conditions. Shuffling must occur across examples while preserving reference count and approximate length, and batched caches must prevent cross-example leakage.

Four families of outcomes should be reported together:

1. **Task quality:** loss, perplexity, calibrated accuracy, or task-specific utility.
2. **Reference and chunk causality:** valid-versus-disabled, valid-versus-shuffled, valid-versus-irrelevant, oracle-chunk, full-reference, and gist-only deltas.
3. **Preservation:** ordinary language and non-reference task quality before and after adaptation.

4. **Resource use:** processed tokens, available versus selected memory tokens, synchronized cache/routing/attention latency, bucket padding ratio, throughput, peak memory, cache bytes, and reuse rate.

Evidence of genuine reference use requires more than a disabled-path gap. A branch can act as a generic domain adapter regardless of supplied content. Valid references should outperform shuffled and irrelevant content, ideally by larger margins on examples proven to be locally insufficient. Reference routing and chunk routing should separately rank known relevant objects and evidence spans above controls. Claims should be repeated across seeds and reported with matched token, parameter, and compute budgets.

15 Research Agenda

The remaining agenda has four decisive tests. First, compare full context, sparse attention, selected-text retrieval, agentic search, and native-state selection on identical natural-task cohorts and token/output budgets. Second, scale logical memory from 32K to at least 8M tokens while reporting task quality, retrieval causality, active and resident bytes, index/backing-store cost, transfer, latency, and reuse. Third, select routing, materialization, and consumer policies on validation identities and evaluate them once on untouched identities across model families. Fourth, exercise concurrent lifecycle behavior—authorization, versioning, cancellation, eviction, restoration, and tenant isolation—under realistic cache pressure. These tests, rather than a paper chronology, determine whether the position survives.

15.1 Falsification Criteria

The research program should be abandoned or substantially revised if repeated, well-controlled studies show any of the following:

- valid references do not outperform shuffled or irrelevant references once formatting and capacity are controlled;
- equivalent prompt concatenation or RAG matches quality at lower end-to-end cost;
- reference encoding and cache management erase any savings from shorter local context, even with reuse;
- learned expansion policies repeatedly fail to match feasible retrieval or cache-selection baselines at the same materialized-token and latency budget; a miss against an unattainable oracle diagnoses the router rather than falsifying the architecture;
- local language ability degrades unacceptably under the adaptation needed to use memory;
- URI and cache isolation cannot be enforced under realistic batching and multi-tenant serving.

Each criterion rejects or narrows the corresponding router, workload, or systems claim. Failure at one scale does not invalidate an exact native-state transport result established under another contract. These criteria make PRA a testable family of systems-and-model hypotheses rather than a label that can absorb any retrieval mechanism after the fact.

16 Risks and Limitations

PRA introduces several risks. First, reference selection can fail silently: if the system does not select the right chunk, the model may answer from incomplete context. Second, URI-addressed memory creates provenance and security questions. A reference table is a powerful interface; poisoning or misaddressing entries can mislead the model. Third, cache construction may move cost rather than eliminate it. Encoding references separately is only beneficial when cache reuse, selective transfer, or reduced prompt length outweighs encoding overhead. Fourth, pretrained integration is family-specific because external K/V must respect layer semantics, position encodings, attention variants, and batching constraints. Paper 2 demonstrates exact adapters for multiple tested families, but not universal compatibility.

Fifth, content-independent branch effects can be mistaken for retrieval. A memory residual may improve a domain simply because it contributes additional activations, even when references are shuffled. Sixth, batching can violate semantic isolation: a physically shared cache must not expose one sequence’s references to another, and empty rows must not inherit projection bias. Seventh, preservation claims can be artifacts of freezing rather than architecture. Eighth, hierarchical expansion can amplify latency variance and create denial-of-service surfaces when references recursively fan out. Ninth, raw summaries can introduce a representation mismatch when compared with projected layer queries; optional summary evidence must inhabit the same contextual layer space. Tenth, implicit prompt heads move historical encoding into a separate cold pass; bounded blocks approximate full historical context, and streaming currently rebuilds the complete head after rollover rather than appending K/V incrementally. Eleventh, CPU residency reduces measured GPU capacity but the current per-chunk offload makes cold construction slower. Finally, stale cache entries create a consistency problem: URI, content version, model/tokenizer/routing fingerprints, and encoded layer memory must remain aligned.

These limitations motivate careful experimental design. PRA should not be judged by a single accuracy number. It should be evaluated through controlled ablations that measure answer accuracy, reference and chunk ranking, expansion depth and budget exhaustion, logical, requested, and materialized fractions, timing decomposition, padding allocation, cache hit ratio, content-specific counterfactual deltas, ordinary-path retention, and cache isolation. The series now addresses tiny-scale capacity, positional continuity, learned semantic routing, exact pretrained retrofit, long-prompt displacement, selective transfer, measured GPU residency, bounded associative discovery, controlled evidence-centered materialization, and prototype adaptive effort. Sparse end-to-end iterative benefit, disclosure generalization, batched asynchronous offload, multi-million-token scaling, and efficiency against strong production baselines remain open.

17 Conclusion

Progressive Retrieval Attention proposes a shift in how foundation models interact with memory. Rather than treating context as a flat token budget to be filled eagerly, PRA treats references as first-class computational objects that can be named, selected, materialized, attended to, expanded, reused, revoked, and audited. The model’s logical context may then be much larger than its active working set, just as a program’s address space can exceed resident memory. Unlike conventional paging, however, neural materialization is approximate and learned; success must be demonstrated, not presumed.

The program’s long-term hypothesis is deliberately stronger than its present evidence: a scalable neural memory architecture may separate local computation from sparse, externally backed,

content-addressed memory; progressively disclose only the native state required by the current computation; adapt retrieval effort to uncertainty; and learn representations that remain useful under this sparse memory interface. No current paper demonstrates that complete architecture.

The long-term vision is a foundation-model memory stack with explicit boundaries among parametric knowledge, working context, episodic history, semantic resources, procedures and tools, and changing world state. URI identity, provenance, freshness, hierarchy, permissions, and cache lifetime become architectural concerns rather than application-side annotations. Protocols such as MCP can expose resources and actions, agents can manage high-level plans, and PRA-like runtimes can test whether repeated read-oriented access can move closer to attention without losing governance.

The strongest existing evidence is diagnostic. Contextual native-state slicing preserves the controlled transport benefit through the split-256 condition (255 addressable source units) while a fixed- k active fraction falls to roughly 5%; scoped position controls and pretrained adapters establish additional mechanism exactness. Yet target coverage is incomplete, semantic and attention geometries differ, better associative traversal does not reliably improve answers, and reduced K/V does not by itself establish lower end-to-end latency. These companion observations support the narrow claim that named logical scope can grow faster than the K/V admitted to one operation. They do not show that PRA is generally better than optimized long context, retrieval, or prefix caching.

The decisive next tests are learned sparse-memory assimilation on untouched cohorts, matched 32K–8M+ scaling, and production serving under realistic reuse, batching, cancellation, eviction, and cache pressure. Low reuse, dense evidence dependence, cheap selected-text injection, frequently changing sources, or routing and transfer costs larger than saved attention are all regimes in which PRA is unlikely to help. In a simple repeated-use model, a published resource is competitive only if

$$\frac{C_{\text{publish}} + C_{\text{index}}}{R} + C_{\text{route}} + C_{\text{transfer}} + C_{\text{sparse}} < C_{\text{matched baseline}}, \quad (24)$$

where R is the realized reuse count. Failure of those matched tests should narrow or reject the corresponding virtual-context claim.

References

- [1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [4] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- [5] Andy Clark and David J. Chalmers. The extended mind. *Analysis*, 58(1):7–19, 1998.

- [6] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.
- [7] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- [8] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [9] Yiran Ding, Li Lina Zhang, Chengruidong Zhang, Yuanyuan Xu, Ning Shang, Jiahang Xu, Fan Yang, and Mao Yang. LongRoPE: Extending LLM context window beyond 2 million tokens. *arXiv preprint arXiv:2402.13753*, 2024.
- [10] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.
- [11] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [12] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.
- [14] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.
- [15] Dongseong Hwang, Weiran Wang, Zhuoyuan Huo, Khe Chai Sim, and Pedro Moreno Mengibar. TransformerFAM: Feedback attention is working memory. *arXiv preprint arXiv:2404.09173*, 2024.
- [16] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
- [17] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020.
- [18] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.

- [20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [21] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [22] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024.
- [23] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.
- [24] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, pages 32332–32344, 2024.
- [25] Model Context Protocol Contributors. Model context protocol specification. <https://modelcontextprotocol.io/specification/2025-06-18/>, 2025. Protocol revision 2025-06-18.
- [26] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. In *Advances in Neural Information Processing Systems*, 2023.
- [27] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.
- [28] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [29] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [31] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems*, 2015.
- [32] Francisco J. Varela, Evan Thompson, and Eleanor Rosch. *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, 1991.

- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. Preprint: <https://arxiv.org/abs/1706.03762>.
- [34] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [35] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.
- [36] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of NAACL-HLT*, 2016.
- [37] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [39] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
- [40] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.